

# Tài Liệu Kỹ Thuật — Hệ Thống Bản Đồ Nhiệt Phát Hiện Tài Xế Đi Chệch Tuyến Đường Thời Gian Thực

Dự án: heat\_map\_pro

Phiên bản: 1.0.0

Cập nhật lần cuối: 23/08/2026

Người thực hiện: Nguyễn Văn Đoàn

## Mục Lục

1. Tổng Quan Dự Án

2. Kiến Trúc Hệ Thống

3. Công Nghệ Sử Dụng

4. Cấu Trúc Thư Mục

5. Backend — Dịch Vụ Go

- 5.1 Điểm Khởi Tạo & Kết Nối Phụ Thuộc
- 5.2 Hệ Thống Cấu Hình
- 5.3 Pipeline Tiếp Nhận Dữ Liệu GPS
- 5.4 Lọc Hai Giai Đoạn
- 5.5 Chỉ Mục Không Gian (Lưới H3)
- 5.6 Bộ Tổng Hợp Không Khóa (Lock-Free)
- 5.7 Phát Hành Qua Redis
- 5.8 Lưu Trữ PostgreSQL
- 5.9 Hub WebSocket Cho Quản Trị Viên
- 5.10 Module Xác Thực
- 5.11 Mẫu Adapter Cho Persistence

6. Lược Đồ Cơ Sở Dữ Liệu

7. Tham Chiếu REST API

8. Giao Thức WebSocket

9. Lược Đồ Protobuf

10. Frontend — Bảng Điều Khiển Quản Trị

11. Frontend — Trình Mô Phỏng Tài Xế

12. Dịch Vụ AI Agent

13. Hạ Tầng & Triển Khai

14. Ràng Buộc Hiệu Năng & Tối Ưu Hóa

15. Biến Môi Trường

16. Quy Trình Phát Triển

17. Chiến Lược Kiểm Thử

## 1. Tổng Quan Dự Án

Hệ Thống Bản Đồ Nhiệt Phát Hiện Tài Xế Đi Chệch Tuyến Đường Thời Gian Thực (Real-Time Driver Deviation Heatmap) là một hệ thống phát hiện, tổng hợp và trực quan hóa các trường hợp tài xế taxi/xe công nghệ đi lệch khỏi tuyến đường đã lên kế hoạch. Hệ thống hoạt động ở hai chế độ:

- Chế độ thời gian thực (Live):** Các tài xế mô phỏng gửi dữ liệu GPS qua WebSocket → backend lọc, đánh chỉ mục không gian, và phát sự kiện lệch tuyến đến bảng điều khiển quản trị trong vòng 1 giây.
- Chế độ lịch sử (History):** Bảng điều khiển truy vấn PostgreSQL để lấy các sự kiện lệch tuyến đã lưu trữ và hiển thị bản đồ nhiệt H3 hexagon tương tác ở nhiều mức zoom (resolution 4–14).

## Các Trường Hợp Sử Dụng Chính

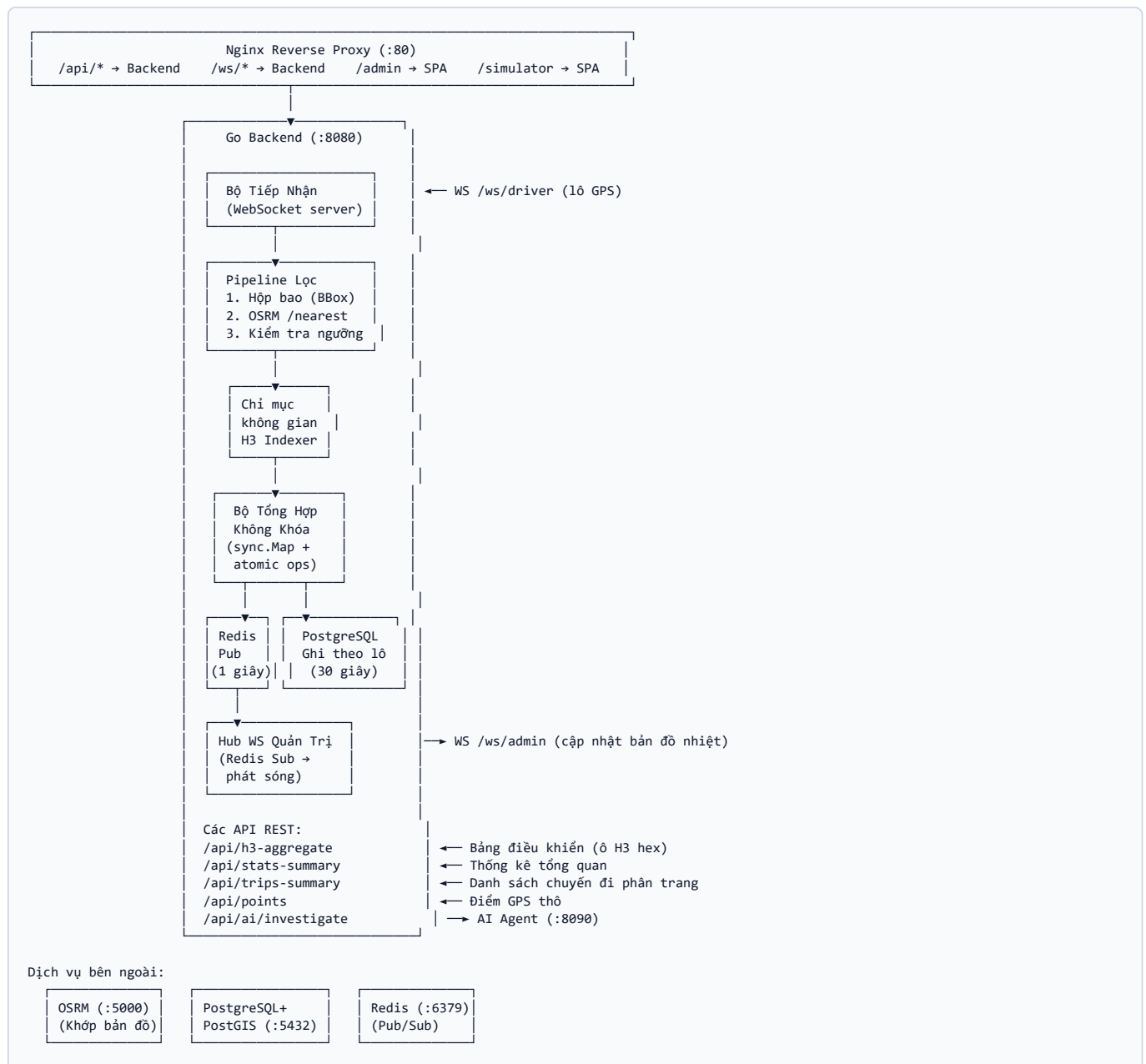
| Trường hợp                  | Tác nhân                 | Mô tả                                                                                             |
|-----------------------------|--------------------------|---------------------------------------------------------------------------------------------------|
| Tiếp nhận GPS               | Trình mô phỏng taxi      | Gửi các lô điểm GPS mỗi 3 giây qua WebSocket                                                      |
| Phát hiện lịch tuyến        | Pipeline Backend         | Lọc GPS qua hộp bao → khớp bản đồ OSRM → kiểm tra ngưỡng                                          |
| Bản đồ nhiệt thời gian thực | Bảng điều khiển quản trị | Hiển thị ô lịch tuyến cập nhật mỗi 1 giây qua WebSocket                                           |
| Phân tích lịch sử           | Bảng điều khiển quản trị | Truy vấn hơn 386k sự kiện lịch tuyến với H3 resolution thích ứng (4–14)                           |
| Điều tra chuyến đi          | Quản trị + AI Agent      | Click vào ô H3 → AI agent thu thập thời tiết, giao thông, hồ sơ taxi và đưa ra chẩn đoán có cơ sở |

## Bộ Dữ Liệu

Hệ thống được phát triển và kiểm thử trên **bộ dữ liệu GPS Taxi Porto** ( [train.csv](#) , ~1.9 GB):

- ~**386.000** sự kiện lịch tuyến lưu trong PostgreSQL
- ~**9.944** chuyến đi duy nhất
- **225** taxi duy nhất
- Phạm vi địa lý: **Porto, Bồ Đào Nha** (vĩ độ ~41.14–41.18, kinh độ ~-8.65—8.57)

## 2. Kiến Trúc Hệ Thống



### Tóm Tắt Luồng Dữ Liệu

1. Trình mô phỏng tài xế → WebSocket (`/ws/driver`) → Bộ Tiếp Nhận
2. Bộ Tiếp Nhận → Lọc hộp bao → OSRM `/nearest` → Kiểm tra ngưỡng
3. Xác nhận lịch tuyến → Chỉ mục H3 (tính toán ô  $O(1)$ ) → Bộ Tổng Hợp (tăng nguyên tử)
4. Redis Publisher (mỗi 1 giây): chụp nhanh bộ tổng hợp → phát JSON lên kênh `heatmap:updates`
5. Hub WebSocket Quản Trị: đăng ký kênh Redis → phát sóng đến tất cả client quản trị đang kết nối
6. PostgreSQL Writer (mỗi 30 giây): chèn hàng loạt các sự kiện đệm vào bảng `deviation_events`
7. Bảng điều khiển quản trị (REST): truy vấn `/api/h3-aggregate` với resolution + bbox viewport → server tính toán ô H3 → trả về polygon sẵn sàng vẽ

### 3. Công Nghệ Sử Dụng

| Tầng                 | Công nghệ                     | Phiên bản | Mục đích                                                            |
|----------------------|-------------------------------|-----------|---------------------------------------------------------------------|
| Backend              | Go                            | 1.22+     | HTTP server, WebSocket, pipeline xử lý dữ liệu                      |
| Chỉ mục không gian   | Lưới H3 tùy chỉnh             | —         | Chỉ mục ô lưới thuần Go (không cần CGO khi dev)                     |
| Cơ sở dữ liệu        | PostgreSQL + PostGIS          | 16 + 3.4  | Lưu trữ bền vững cho sự kiện lịch tuyến, chuyến đi, tài xế          |
| Hàng đợi tin nhắn    | Redis                         | 7         | Pub/Sub cho cập nhật bản đồ nhiệt thời gian thực (1 giây/lần)       |
| Khớp bản đồ          | OSRM                          | mới nhất  | Bắt dính điểm GPS vào mạng lưới đường qua <code>/nearest</code> API |
| Frontend Quản Trị    | React 18 + Vite               | —         | Bảng điều khiển bản đồ nhiệt tương tác với MapLibre GL JS           |
| Frontend Mô Phỏng    | React 18 + Vite               | —         | Mô phỏng GPS tài xế với WebSocket streaming                         |
| Hiển thị bản đồ      | MapLibre GL JS                | —         | Trình hiển thị bản đồ mã nguồn mở (không cần API key)               |
| AI Agent             | Python + FastAPI              | —         | Công cụ điều tra theo ngữ cảnh (mẫu ReAct)                          |
| Mô hình ngôn ngữ lớn | Groq (LLaMA 3.3 70B) / Gemini | —         | Tạo chẩn đoán có căn cứ                                             |
| Reverse Proxy        | Nginx                         | alpine    | Điểm vào duy nhất, phục vụ file tĩnh, nâng cấp WebSocket            |
| Tuần tự hóa          | Protocol Buffers v3           | —         | Định nghĩa lược đồ (nguồn chính xác duy nhất cho tin nhắn)          |
| Container hóa        | Docker Compose                | —         | Điều phối toàn bộ stack với health check                            |

#### Thư Viện Chính

##### Backend Go:

- `github.com/gorilla/websocket` — Server WebSocket
- `github.com/jackc/pgx/v5` — Driver PostgreSQL với connection pooling
- `github.com/redis/go-redis/v9` — Client Redis
- `log/slog` — Ghi log có cấu trúc JSON (thư viện chuẩn)
- `sync.Map` + `sync/atomic` — Cơ chế đồng thời không khóa

##### Frontend (Quản Trị + Mô Phỏng):

- `maplibre-gl` — Hiển thị bản đồ với tile CARTO dark-matter
- `protobufjs` — Giải mã Protobuf (dự kiến, hiện tại dùng JSON)

## 4. Cấu Trúc Thư Mục

```
heat_map_pro/
├── .agents/                # Quy tắc cho AI agent (AGENTS.md)
├── .env                    # Biến môi trường (không đưa lên git)
├── .env.example            # Mẫu biến môi trường có chú thích
├── Makefile                # Lệnh build/dev/test gốc
├── package.json            # Script khởi động monorepo (concurrently)
├── proto/
│   └── heatmap/v1/
│       └── messages.proto  # ★ Nguồn chính xác duy nhất cho mọi tin nhắn
├── backend/
│   ├── Dockerfile          # Build Go đa giai đoạn (alpine)
│   ├── go.mod / go.sum     # Go module (1.22+)
│   ├── cmd/
│   │   └── server/
│   │       └── main.go     # ★ Điểm khởi tạo: kết nối phụ thuộc & khởi động
│   ├── gen/
│   │   └── heatmap/v1/     # Code Go được tạo tự động từ Protobuf
│   └── internal/
│       ├── config/         # Struct cấu hình & đọc biến môi trường
│       ├── ingestion/      # Bộ xử lý WebSocket + pipeline lọc
│       ├── filter/         # Hộp bao + client OSRM
│       ├── spatial/        # Chỉ mục lưới H3 (thuần Go)
│       ├── aggregator/     # Bộ đếm không khóa (sync.Map + atomic)
│       ├── publisher/      # Phát hành Redis + flush theo lô
│       ├── persistence/    # Ghi PostgreSQL + các handler REST API
│       ├── websocket/      # Hub WebSocket quản trị (Redis sub → phát sóng)
│       └── auth/           # Xác thực tài xế (đăng ký/đăng nhập/xem hồ sơ)
├── frontend/
│   ├── admin/              # Bảng Điều Khiển Quản Trị (React + Vite)
│   │   └── src/
│   │       ├── App.jsx     # Gốc: chuyển chế độ, tải dữ liệu, chọn chuyển đi
│   │       ├── components/
│   │       │   ├── MapContainer.jsx # Bọc bản đồ MapLibre GL
│   │       │   ├── HeatmapLayer.jsx # ★ Vẽ H3 hex, đùn 3D, logic fetch
│   │       │   ├── FilterPanel.jsx  # Bộ lọc ngày/tài xế/chuyển đi + điều tra AI
│   │       │   ├── StatsOverlay.jsx # Lớp phủ thống kê live/ lịch sử
│   │       │   ├── DriverList.jsx   # Danh sách tài xế với lựa chọn
│   │       │   ├── HourlyAnalyticsChart.jsx # Biểu đồ phân tích né tránh 24 giờ
│   │       │   └── ToastNotification.jsx # Thông báo chuyển đi mới
│   │       ├── hooks/
│   │       │   ├── useHeatmapStream.js # Hook WebSocket cho bản đồ nhiệt live
│   │       │   └── useProgressiveData.js # ★ Tải dữ liệu tiến triển 3 pha
│   │       ├── utils/
│   │       │   └── osrmRouting.js     # Khớp tuyến OSRM & chồng lấp H3
│   │       └── workers/
│   │           └── h3Worker.js        # Web Worker cho tính toán H3
│   └── simulator/              # Trình Mô Phỏng Tài Xế (React + Vite)
│       └── src/
│           ├── App.jsx        # Mô phỏng GPS, vẽ tuyến, lưu chuyển đi
│           ├── components/    # ControlPanel, MapView, StatsBar, AuthModal
│           ├── hooks/
│           │   └── useWebSocket.js # Hook WebSocket dùng chung
│           ├── lib/
│           │   └── routeService.js # Lấy tuyến đường OSRM
│           ├── workers/
│           │   └── simulationWorker.js # Web Worker cho tạo GPS
├── ai/
│   ├── agent/                # Dịch Vụ AI Agent (Python)
│   │   ├── Dockerfile        # Build Python
│   │   ├── main.py           # Server FastAPI (:8090)
│   │   ├── models.py         # Mô hình dữ liệu Pydantic
│   │   ├── react_engine.py    # Công cụ điều tra ReAct
│   │   ├── llm_client.py      # Tích hợp LLM (Groq/Gemini)
│   │   └── tools/             # Các hàm công cụ
│   │       ├── weather.py     # API thời tiết (Open-Meteo)
│   │       ├── reverse_geocode.py # Chuyển tọa độ ngược (Nominatim)
│   │       ├── news_search.py # Tìm kiếm sự cố
│   │       ├── db_telemetry.py # Truy vấn telemetry PostgreSQL
│   │       ├── osrm_alternatives.py # Phân tích tuyến thay thế OSRM
│   │       ├── driver_profile.py # Lịch sử tuần thủ tài xế
│   │       └── traffic_speed_ratio.py # Phân tích tốc độ giao thông
├── infra/
│   ├── docker-compose.yml     # ★ Stack production đầy đủ (7 dịch vụ)
│   ├── docker-compose.dev.yml # Ghi đè cho phát triển
│   ├── nginx/
│   │   └── nginx.conf         # Cấu hình reverse proxy
│   ├── osrm/                  # Dữ liệu bản đồ OSRM (không đưa lên git)
│   └── postgres/
│       ├── init.sql           # ★ Lược đồ CSDL + chỉ mục + hàm
│       └── 02-ai-columns.sql  # Bổ sung cột cho AI
```

## 5. Backend — Dịch Vụ Go

### 5.1 Điểm Khởi Tạo & Kết Nối Phụ Thuộc

File: `main.go`

Điểm khởi tạo tuân theo mẫu **composition root** — đây là file DUY NHẤT import tất cả package nội bộ và kết nối chúng lại. Không có logic nghiệp vụ nào ở đây.

Trình tự khởi động:

1. Tải cấu hình từ biến môi trường
2. Khởi tạo logger có cấu trúc JSON ( `log/slog` )
3. Tạo context gốc với cancel (để tắt máy chủ an toàn)
4. Khởi tạo tất cả phụ thuộc theo thứ tự:
  - `spatial.NewH3Indexer(resolution)` — Lưới H3
  - `aggregator.New()` — Bộ đếm không khóa
  - `filter.NewOSRMClient(url, timeout)` — Client HTTP OSRM
  - `filter.NewBoundingBoxFilter(bufferMeters)` — Hộp bao trong bộ nhớ
  - `publisher.NewRedisPublisher(ctx, cfg)` — Kết nối Redis
  - `persistence.NewPostgresWriter(ctx, cfg)` — Pool PostgreSQL
  - `websocket.NewHub(ctx, cfg)` — Hub WebSocket quản trị
  - `persistence.NewAdapter(pgWriter)` — Cầu nối adapter
  - `ingestion.NewHandler(...)` — Pipeline tiếp nhận
  - `auth.NewRepository/NewHandler` — Xác thực
5. Kết nối phụ thuộc chéo (bộ đếm tài xế, callback lưu chuyển đi)
6. Khởi động 3 goroutine nền:
  - Vòng lặp flush Redis (mỗi 1 giây)
  - Vòng lặp flush PostgreSQL (mỗi 30 giây)
  - Đăng ký WebSocket subscriber (Redis → phát sóng quản trị)
7. Đăng ký các route HTTP trên `http.ServeMux`
8. Khởi động HTTP server với middleware CORS + Gzip
9. Chờ SIGINT/SIGTERM → tắt máy chủ an toàn

Chuỗi middleware: `corsHandler → gzipHandler → mux`

- CORS: `Access-Control-Allow-Origin: *` (mở cho demo)
- Gzip: `gzip.BestSpeed` cho tất cả response không phải WebSocket (~80% nén)
- Các đường WebSocket ( `/ws/*` ) bỏ qua gzip để tránh xung đột upgrade

Cấu hình HTTP Server:

| Tham số                   | Giá trị | Lý do                            |
|---------------------------|---------|----------------------------------|
| <code>ReadTimeout</code>  | 15 giây | Chống tấn công client chậm       |
| <code>WriteTimeout</code> | 30 giây | H3 aggregate quét 386k dòng      |
| <code>IdleTimeout</code>  | 60 giây | Keep-alive cho dashboard polling |

5.2 Hệ Thống Cấu Hình

File: `config.go`

Mọi cấu hình được tải từ biến môi trường với giá trị mặc định hợp lý. Trình đọc file `.env` tùy chỉnh hỗ trợ định dạng dotenv (không phụ thuộc thư viện ngoài).

Các trường trong struct Config:

| Trường                                | Biến môi trường                         | Mặc định                           | Mô tả                                          |
|---------------------------------------|-----------------------------------------|------------------------------------|------------------------------------------------|
| <code>AppEnv</code>                   | <code>APP_ENV</code>                    | <code>development</code>           | Chế độ môi trường                              |
| <code>LogLevel</code>                 | <code>LOG_LEVEL</code>                  | <code>debug</code>                 | Mức độ chi tiết log                            |
| <code>BackendPort</code>              | <code>BACKEND_PORT</code>               | <code>8080</code>                  | Cổng lắng nghe HTTP/WS                         |
| <code>OSRMURL</code>                  | <code>OSRM_URL</code>                   | <code>http://127.0.0.1:5000</code> | URL gốc OSRM                                   |
| <code>OSRMMatchTimeoutMS</code>       | <code>OSRM_MATCH_TIMEOUT_MS</code>      | <code>500</code>                   | Timeout gọi OSRM (ms)                          |
| <code>RedisAddr</code>                | <code>REDIS_ADDR</code>                 | <code>127.0.0.1:6379</code>        | Redis host:port                                |
| <code>RedisChannel</code>             | <code>REDIS_CHANNEL</code>              | <code>heatmap:updates</code>       | Tên kênh Pub/Sub                               |
| <code>PostgresHost</code>             | <code>POSTGRES_HOST</code>              | <code>127.0.0.1</code>             | Host PostgreSQL                                |
| <code>PostgresPort</code>             | <code>POSTGRES_PORT</code>              | <code>5432</code>                  | Cổng PostgreSQL                                |
| <code>PostgresUser</code>             | <code>POSTGRES_USER</code>              | <code>heatmap</code>               | Tên người dùng CSDL                            |
| <code>PostgresPassword</code>         | <code>POSTGRES_PASSWORD</code>          | <code>heatmap_secret_2024</code>   | Mật khẩu CSDL                                  |
| <code>PostgresDB</code>               | <code>POSTGRES_DB</code>                | <code>heatmap_db</code>            | Tên cơ sở dữ liệu                              |
| <code>H3Resolution</code>             | <code>H3_RESOLUTION</code>              | <code>8</code>                     | Resolution H3 mặc định (chế độ thời gian thực) |
| <code>FlushIntervalRedisMS</code>     | <code>FLUSH_INTERVAL_REDIS_MS</code>    | <code>1000</code>                  | Chu kỳ flush Redis                             |
| <code>FlushIntervalPostgresS</code>   | <code>FLUSH_INTERVAL_POSTGRES_S</code>  | <code>30</code>                    | Chu kỳ flush PostgreSQL                        |
| <code>BBoxBufferMeters</code>         | <code>BBOX_BUFFER_METERS</code>         | <code>50</code>                    | Khoảng cách mở rộng hộp bao                    |
| <code>DeviationThresholdMeters</code> | <code>DEVIATION_THRESHOLD_METERS</code> | <code>50</code>                    | Ngưỡng tối thiểu để đánh dấu lệch tuyến        |

Kiểm tra hợp lệ: H3 resolution (0–15), phạm vi cổng (1–65535).

Tạo DSN: `postgres://user:pass@host:port/db?sslmode=disable`

5.3 Pipeline Tiếp Nhận Dữ Liệu GPS

File: `ingestion/handler.go`

Bộ tiếp nhận quản lý các kết nối WebSocket từ trình mô phỏng tài xế và xử lý điểm GPS qua pipeline 6 giai đoạn.

**Thiết kế interface-first:** Tất cả phụ thuộc được định nghĩa dưới dạng interface trong file này (package người tiêu dùng), không phải trong package nhà cung cấp:

```

type Filter interface {
    IsInsideBBox(lat, lng float64, tripID string) bool
    RegisterTrip(tripID string, waypoints []Waypoint) BoundingBox
}

type MapMatcher interface {
    MatchAndDistance(ctx context.Context, lat, lng float64, tripWaypoints []Waypoint) (float64, error)
}

type SpatialIndexer interface {
    LatLngToCell(lat, lng float64) string
}

type DeviationAggregator interface {
    Increment(h3Index string)
}

type EventPersister interface {
    BufferEvent(event DeviationEventData)
}

```

### Pipeline xử lý GPS (mỗi điểm):

```

Giai đoạn 1: Kiểm tra Hộp Bao (O(1), trong bộ nhớ)
├── Nằm trong bbox → BỎ QUA (điểm nằm trên tuyến kế hoạch)
└──
↓
Giai đoạn 2: Khớp Bản Đồ OSRM (HTTP, timeout 500ms)
├── Lỗi → BỎ QUA với cảnh báo
└──
↓
Giai đoạn 3: Kiểm tra Ngưỡng (khoảng cách ≤ 50m)
├── Dưới ngưỡng → BỎ QUA (nhiều GPS)
└──
↓
Giai đoạn 4: Chỉ Mục Không Gian H3 (O(1), toán thuần)
└──
↓
Giai đoạn 5: Tổng Hợp Nguyên Tử (LOCK XADD, không tranh chấp)
└──
↓
Giai đoạn 6: Đệm cho PostgreSQL (ghi theo lô mỗi 30 giây)

```

**Định dạng tin nhắn:** Hiện tại là JSON ( `GPSBatch` từ lược đồ Protobuf), sẽ chuyển sang Protobuf nhị phân để tin nhắn nhỏ hơn ~10 lần trong production.

**Theo dõi tài xế hoạt động:** `sync.RWMutex` bảo vệ `map[string]bool`, truy cập qua `ActiveDriverCount()` cho endpoint `/api/health`.

## 5.4 Lọc Hai Giai Đoạn

### Giai đoạn 1: Lọc Trước Bằng Hộp Bao (Bounding Box)

File: `filter/bounding_box.go`

- Kiểm tra chứa O(1):  $lat \in [minLat, maxLat] \wedge lng \in [minLng, maxLng]$
- Hộp bao được mở rộng thêm `BBOX_BUFFER_METERS` (mặc định 50m) theo mọi hướng
- Chuyển đổi mét sang độ:  $1^\circ vĩ \approx 111.320m$ ,  $1^\circ kinh \approx 111.320 \times \cos(vĩ) m$
- An toàn luồng: `sync.RWMutex` bảo vệ `map trip→bbox`

**Mục đích:** Loại bỏ ~80% điểm GPS mà không cần gọi HTTP, giảm đáng kể tải OSRM.

### Giai đoạn 2: Khớp Bản Đồ OSRM

File: `filter/osrm_client.go`

- Gọi OSRM `/nearest/v1/driving/{lng},{lat}?number=1`
- Trả về khoảng cách bắt dính (mét từ điểm GPS đến đường gần nhất)
- Nếu có waypoint chuyển đi, tính khoảng cách Haversine đến waypoint gần nhất
- Client HTTP: timeout 500ms, connection pooling (10 kết nối nhàn rỗi mỗi host)
- Công thức Haversine:  $d = R \times c$ , trong đó  $c = 2 \times \text{atan2}(va, \sqrt{1-a})$  và  $R = 6.371.000 m$

## 5.5 Chỉ Mục Không Gian (Lưới H3)

File: `spatial/h3_indexer.go`

**Ghi chú triển khai:** Đây là lưới geohash thuần Go (không phụ thuộc CGO). Trong production, thay thế bằng `uber/h3-go` để có chỉ mục hexagon H3 thực sự.

Tính toán ô: `LatLngToCell(lat, lng) → "H{res}:{latGrid}:{lngGrid}"`

```
latGrid = floor((lat + 90) / cellSizeDeg)
lngGrid = floor((lng + 360*_đă_chuẩn_hóa) / cellSizeDeg)
```

Bảng Resolution → Kích Thước Ô:

| Resolution | Kích thước ô (°) | Đường kính xấp xỉ | Trường hợp sử dụng               |
|------------|------------------|-------------------|----------------------------------|
| 4          | 0,1326           | ~14,7 km          | Xem toàn quốc                    |
| 5          | 0,0500           | ~5,6 km           | Xem vùng                         |
| 6          | 0,0189           | ~2,1 km           | Xem thành phố                    |
| 7          | 0,00713          | ~793 m            | Xem quận                         |
| 8          | 0,00414          | ~460 m            | <b>Mặc định</b> (thời gian thực) |
| 9          | 0,00156          | ~174 m            | Xem khu vực                      |
| 10         | 0,000589         | ~66 m             | Xem cấp khối                     |
| 11         | 0,000222         | ~25 m             | Xem cấp đường                    |
| 12         | 0,0000838        | ~9,3 m            | Xem ngã tư                       |
| 13         | 0,0000316        | ~3,5 m            | Xem cấp làn đường                |
| 14         | 0,0000119        | ~1,3 m            | Chi tiết tối đa                  |

Các phương thức bổ sung:

- `CellToLatLng()` — ngược lại: chỉ mục ô → tọa độ tâm
- `CellToBoundary()` — tạo đa giác hexagon 6 đỉnh (hướng đỉnh phẳng, co giãn kinh độ theo vĩ độ)
- `CellSizeDeg()` — trả về kích thước ô để phía client vẽ hexagon

## 5.6 Bộ Tổng Hợp Không Khóa (Lock-Free)

File: `aggregator/lockfree.go`

Bộ tổng hợp là **đường nóng** (hot path) của hệ thống — mọi lệnh tuyến được xác nhận đều tăng bộ đếm ở đây. Nó phải xử lý ghi đồng thời từ nhiều goroutine WebSocket mà không bị tranh chấp khóa.

Cấu trúc dữ liệu: `sync.Map[string]*uint64`

Các thao tác:

- `Increment(h3Index)`: `sync.Map.LoadOrStore()` + `atomic.AddUint64()` — biên dịch thành một lệnh CPU `LOCK XADD` duy nhất
- `Snapshot()`: hoán đổi nguyên tử tất cả bộ đếm về 0 và trả về giá trị trước hoán đổi. Được gọi bởi vòng lặp flush Redis mỗi 1 giây. Không chặn: các goroutine khác tiếp tục tăng trong khi duyệt.
- `TotalActive()`: đếm số ô có bộ đếm khác 0 (xấp xỉ, không nhất quán nguyên tử giữa các ô)

Quy tắc thiết kế: KHÔNG dùng `sync.Mutex` trong package này. Chỉ dùng `sync.Map` + thao tác `atomic`.

## 5.7 Phát Hành Qua Redis

File: `publisher/redis_pub.go`

Vòng lặp flush (mỗi 1 giây):

1. Chụp nhanh từ bộ tổng hợp (hoán đổi nguyên tử về 0)
2. Bỏ qua nếu ảnh chụp rỗng (không có lịch tuyến trong 1 giây qua)
3. Xây dựng `heatmapUpdateJSON` với các ô, timestamp, số tài xế, tổng lịch tuyến
4. `json.Marshal()` và `PUBLISH` lên kênh Redis `heatmap:updates`

Định dạng JSON được phát hành:

```
{
  "cells": [
    { "h3_index": "H8:2602:25773", "intensity": 5, "last_updated": 1724389200000 }
  ],
  "server_timestamp": 1724389200000,
  "total_drivers": 12,
  "total_deviations": 47
}
```

Bổ sung: `PublishRaw()` phát hành payload JSON thô tùy ý (dùng cho sự kiện `new_trip`).

## 5.8 Lưu Trữ PostgreSQL

File: `persistence/postgres.go` (1.444 dòng)

Pool kết nối: `pgxpool` với `MaxConns=5`, `MinConns=1`.

Bộ ghi theo lô:

- Các sự kiện được đệm trong slice bảo vệ bởi `sync.Mutex` (dung lượng ban đầu: 1024)
- Vòng lặp flush chạy mỗi 30 giây
- Sử dụng `pgx.Batch` để chèn nhiều dòng hiệu quả

Các handler REST API được export từ package này:

| Handler                              | Route                               | Mô tả                                                                        |
|--------------------------------------|-------------------------------------|------------------------------------------------------------------------------|
| <code>HandleSaveTrip</code>          | <code>POST /api/trips</code>        | Lưu chuyến đi từ trình mô phỏng                                              |
| <code>HandleGetTrips</code>          | <code>GET /api/trips</code>         | Lấy danh sách chuyến đi (lọc theo tài xế/giới hạn)                           |
| <code>HandleTripsSummaryQuery</code> | <code>GET /api/trips-summary</code> | Tóm tắt chuyến đi phân trang (tổng hợp từ <code>deviation_events</code> )    |
| <code>HandleStatsSummary</code>      | <code>GET /api/stats-summary</code> | Thống kê toàn bộ dataset (tức thì, không hardcoded ngày)                     |
| <code>HandleHistoryQuery</code>      | <code>GET /api/history</code>       | Dữ liệu bản đồ nhiệt lịch sử (dùng hàm <code>get_heatmap_for_period</code> ) |
| <code>HandleDeviationsQuery</code>   | <code>GET /api/deviations</code>    | Sự kiện lệch tuyến của tài xế                                                |
| <code>HandlePointsQuery</code>       | <code>GET /api/points</code>        | Điểm GPS thô cho hiển thị bản đồ nhiệt                                       |
| <code>HandleTrajectoriesQuery</code> | <code>GET /api/trajectories</code>  | Quỹ đạo GPS mỗi chuyến đi dưới dạng GeoJSON LineString                       |
| <code>HandleRoadStatsQuery</code>    | <code>GET /api/road-stats</code>    | Thống kê đoạn đường (nhân tiếng Việt)                                        |
| <code>HandleHourlyStatsQuery</code>  | <code>GET /api/hourly-stats</code>  | Thống kê né tránh theo giờ cho biểu đồ 24 giờ                                |
| <code>HandleActualPathQuery</code>   | <code>GET /api/actual-path</code>   | Ô H3 đường thực tế (đường tài xế chọn khi đi lệch)                           |
| <code>HandleH3Aggregate</code>       | <code>GET /api/h3-aggregate</code>  | <b>Tổng hợp H3 phía server</b> (xem phần tiếp)                               |

Tổng Hợp H3 Phía Server

File: `persistence/h3_aggregate.go`

Đây là endpoint quan trọng nhất cho chế độ lịch sử của bảng điều khiển quản trị. Thay vì gửi 386k điểm GPS thô đến trình duyệt, nó:

1. Truy vấn `deviation_events` với khoảng thời gian + bbox tùy chọn + bộ lọc tài xế
2. Cho mỗi dòng, tính `LatLngToCell(lat, lng)` tại resolution yêu cầu
3. Tổng hợp theo ô: count, unique\_drivers, avoid\_trips, total\_trips, avg/max deviation
4. Trả về JSON với tọa độ tâm ô + cell\_size (frontend vẽ hexagon cục bộ)

**Bộ nhớ đệm:** `sync.Map` với TTL 60 giây, khóa bằng SHA-256 của tham số truy vấn (resolution, bbox làm tròn 2 chữ số, khoảng thời gian, driver\_id). Cache HIT trả về trong <1ms.

**Định dạng phản hồi:**

```
{
  "cells": [
    {
      "cell": "H12:1565078:4193174",
      "cell_size": 0.0000838,
      "center_lat": 41.153578,
      "center_lng": -8.610042,
      "count": 27,
      "unique_drivers": 24,
      "avoid_trips": 5,
      "total_trips": 18,
      "avg_dev": 125.3,
      "max_dev": 450.7,
      "height": 69,
      "ratio": 0.278
    }
  ],
  "total_cells": 31589,
  "resolution": 12,
  "total_points_processed": 183731
}
```

## 5.9 Hub WebSocket Cho Quản Trị Viên

File: `websocket/hub.go`

- Quản lý tập hợp client WebSocket quản trị đang kết nối (`map[*websocket.Conn]bool`)
- Goroutine nền đăng ký kênh Redis `heatmap:updates`
- Khi nhận tin nhắn Redis, phát sóng JSON thô đến TẤT CẢ client quản trị đang kết nối
- Map client an toàn luồng với `sync.RWMutex`
- Nâng cấp: `gorilla/websocket.Upgrader` với `CheckOrigin: true` (chấp nhận mọi nguồn)

## 5.10 Module Xác Thực

Thư mục: `backend/internal/auth/`

- `auth.go` — Struct `Repository`: truy vấn PostgreSQL cho CRUD tài xế
- `handler.go` — Các handler HTTP:
  - `POST /api/auth/register` — Tạo tài khoản tài xế (email, mật khẩu, tên, SĐT, xe)
  - `POST /api/auth/login` — Xác thực và trả về token JWT/session
  - `GET /api/auth/me` — Trả về hồ sơ tài xế hiện tại

Băm mật khẩu sử dụng `golang.org/x/crypto/bcrypt`.

## 5.11 Mẫu Adapter Cho Persistence

File: `persistence/adapter.go`

Struct `Adapter` làm cầu nối giữa interface `ingestion.EventPersister` và `persistence.PostgresWriter` mà không tạo import vòng:

```

ingestion → (định nghĩa interface EventPersister)
persistence → (triển khai PostgresWriter.BufferEvent)
persistence.Adapter → (chuyển đổi ingestion.DeviationEventData → persistence.DeviationEvent)
cmd/server/main.go → tạo Adapter, truyền cho ingestion.NewHandler

```

## 6. Lược Đồ Cơ Sở Dữ Liệu

File: `infra/postgres/init.sql`

Bảng: `deviation_events`

Bảng lưu trữ chính. Mỗi dòng đại diện cho một điểm GPS được xác nhận là lệch tuyến (>50m so với tuyến kế hoạch).

| Cột                           | Kiểu                          | Mô tả                               |
|-------------------------------|-------------------------------|-------------------------------------|
| <code>id</code>               | <code>BIGSERIAL PK</code>     | Khóa chính tự tăng                  |
| <code>driver_id</code>        | <code>VARCHAR(64)</code>      | Mã tài xế duy nhất                  |
| <code>trip_id</code>          | <code>VARCHAR(128)</code>     | Mã chuyến đi                        |
| <code>latitude</code>         | <code>DOUBLE PRECISION</code> | Vĩ độ WGS84                         |
| <code>longitude</code>        | <code>DOUBLE PRECISION</code> | Kinh độ WGS84                       |
| <code>h3_index</code>         | <code>VARCHAR(20)</code>      | Chỉ mục ô H3 đã tính trước          |
| <code>deviation_meters</code> | <code>DOUBLE PRECISION</code> | Khoảng cách từ tuyến kế hoạch (mét) |
| <code>heading</code>          | <code>REAL</code>             | Hướng đi (0–360°)                   |
| <code>speed_kmh</code>        | <code>REAL</code>             | Tốc độ (km/h)                       |
| <code>created_at</code>       | <code>TIMESTAMP TZ</code>     | Thời điểm sự kiện (mặc định: NOW()) |

Chỉ mục (7 tổng cộng):

| Chỉ mục                                      | Loại  | Cột                                        | Mục đích                              |
|----------------------------------------------|-------|--------------------------------------------|---------------------------------------|
| <code>idx_deviation_events_created_at</code> | BTree | <code>created_at DESC</code>               | Truy vấn theo khoảng thời gian        |
| <code>idx_deviation_events_h3_index</code>   | BTree | <code>h3_index</code>                      | Tổng hợp ô H3                         |
| <code>idx_deviation_events_driver_id</code>  | BTree | <code>driver_id, created_at DESC</code>    | Truy vấn theo tài xế                  |
| <code>idx_deviation_events_h3_time</code>    | BTree | <code>h3_index, created_at DESC</code>     | Truy vấn chính của bảng điều khiển    |
| <code>idx_dev_events_brin_time</code>        | BRIN  | <code>created_at</code> (32 trang/phạm vi) | Quét tuần tự phạm vi (dung lượng nhỏ) |
| <code>idx_dev_events_deviation</code>        | BTree | <code>deviation_meters DESC</code>         | Lấy mẫu Top-N lệch tuyến              |
| <code>idx_dev_events_lat_lng</code>          | BTree | <code>latitude, longitude</code>           | Truy vấn bbox viewport (zoom ≥ 11)    |

Bảng: `trips`

| Cột                                                 | Kiểu                          | Mô tả                                        |
|-----------------------------------------------------|-------------------------------|----------------------------------------------|
| <code>trip_id</code>                                | <code>VARCHAR(128) PK</code>  | Mã chuyến đi duy nhất                        |
| <code>driver_id</code>                              | <code>VARCHAR(64)</code>      | Tài xế thực hiện chuyến đi                   |
| <code>bbox_min/max_lat/lng</code>                   | <code>DOUBLE PRECISION</code> | Hộp bao của tuyến kế hoạch                   |
| <code>origin_json</code>                            | <code>JSONB</code>            | Metadata điểm xuất phát                      |
| <code>destination_json</code>                       | <code>JSONB</code>            | Metadata điểm đến                            |
| <code>waypoints_json</code>                         | <code>JSONB</code>            | Các điểm tuyến kế hoạch                      |
| <code>actual_route_json</code>                      | <code>JSONB</code>            | Quỹ đạo GPS thực tế                          |
| <code>distance_km</code>                            | <code>DOUBLE PRECISION</code> | Quãng đường chuyến đi                        |
| <code>duration_min</code>                           | <code>INTEGER</code>          | Thời gian chuyến đi                          |
| <code>is_deviated</code>                            | <code>BOOLEAN</code>          | Tài xế có đi lệch hay không                  |
| <code>status</code>                                 | <code>VARCHAR(20)</code>      | <code>active</code> / <code>completed</code> |
| <code>created_at</code> / <code>completed_at</code> | <code>TIMESTAMPZ</code>       | Dấu thời gian                                |

Bảng: `drivers`

| Cột                        | Kiểu                             | Mô tả                                        |
|----------------------------|----------------------------------|----------------------------------------------|
| <code>id</code>            | <code>BIGSERIAL PK</code>        | ID tự tăng                                   |
| <code>driver_id</code>     | <code>VARCHAR(64) UNIQUE</code>  | Mã tài xế cấp nghiệp vụ                      |
| <code>email</code>         | <code>VARCHAR(128) UNIQUE</code> | Email đăng nhập                              |
| <code>password_hash</code> | <code>VARCHAR(255)</code>        | Hash bcrypt                                  |
| <code>full_name</code>     | <code>VARCHAR(128)</code>        | Tên hiển thị                                 |
| <code>phone</code>         | <code>VARCHAR(32)</code>         | Số điện thoại                                |
| <code>license_plate</code> | <code>VARCHAR(32)</code>         | Biển số xe                                   |
| <code>vehicle_type</code>  | <code>VARCHAR(32)</code>         | Mặc định: <code>taxi</code>                  |
| <code>status</code>        | <code>VARCHAR(20)</code>         | <code>active</code> / <code>suspended</code> |

View: `heatmap_summary`

View dựng sẵn cho truy vấn tổng hợp thường dùng trên bảng điều khiển:

```
SELECT h3_index, COUNT(*)::INTEGER AS intensity,
       MAX(created_at) AS last_updated,
       COUNT(DISTINCT driver_id) AS unique_drivers
FROM deviation_events GROUP BY h3_index;
```

Hàm: `get_heatmap_for_period(p_from, p_to)`

Trả về dữ liệu ô H3 đã tổng hợp cho một khoảng thời gian. Được sử dụng bởi `GET /api/history`.

7. Tham Chiếu REST API

URL gốc: `http://localhost:8080`

Sức Khỏe & Thống Kê

| Phương thức | Endpoint           | Mô tả                                 | Phản hồi                                                                    |
|-------------|--------------------|---------------------------------------|-----------------------------------------------------------------------------|
| GET         | /api/health        | Sức khỏe server + thời gian hoạt động | { status, uptime_seconds, active_drivers }                                  |
| GET         | /api/stats-summary | Thống kê toàn bộ dataset              | { total_points, total_trips, total_drivers, data_from_ms, data_to_ms, ... } |

Quản Lý Chuyến Đi

| Phương thức | Endpoint           | Tham số                                                           | Mô tả                                                       |
|-------------|--------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| POST        | /api/trips         | Body: { trip_id, driver_id, waypoints, origin, destination, ... } | Lưu chuyến đi từ trình mô phỏng                             |
| GET         | /api/trips         | ?driver_id=&limit=500                                             | Lấy bản ghi chuyến đi                                       |
| GET         | /api/trips-summary | ?page=1&page_size=200&from=&to=&driver_id=                        | Tóm tắt chuyến đi phân trang (tổng hợp từ deviation_events) |

Dữ Liệu Bản Đồ Nhiệt

| Phương thức | Endpoint          | Tham số                                                                 | Mô tả                                                   |
|-------------|-------------------|-------------------------------------------------------------------------|---------------------------------------------------------|
| GET         | /api/h3-aggregate | ?from=&to=&resolution=12&min_lat=&max_lat=&min_lng=&max_lng=&driver_id= | Endpoint chính: Tổng hợp H3 phía server                 |
| GET         | /api/history      | ?from=&to=                                                              | Bản đồ nhiệt lịch sử (dùng hàm get_heatmap_for_period ) |
| GET         | /api/points       | ?from=&to=&limit=50000                                                  | Điểm GPS lịch tuyến thô                                 |
| GET         | /api/actual-path  | ?from=&to=                                                              | Ô H3 đường thực tế (tuyến tài xế đi khi lịch)           |

Phân Tích

| Phương thức | Endpoint          | Tham số               | Mô tả                                        |
|-------------|-------------------|-----------------------|----------------------------------------------|
| GET         | /api/deviations   | ?driver_id=&from=&to= | Sự kiện lệch tuyến cá nhân                   |
| GET         | /api/trajectories | ?trip_id=             | GeoJSON LineString cho quỹ đạo GPS chuyến đi |
| GET         | /api/road-stats   | ?lat=&lng=&radius=    | Thống kê đoạn đường (nhãn tiếng Việt)        |
| GET         | /api/hourly-stats | ?from=&to=            | Phân bố né tránh theo giờ (biểu đồ 24 giờ)   |

Xác Thực

| Phương thức | Endpoint           | Mô tả                     |
|-------------|--------------------|---------------------------|
| POST        | /api/auth/register | Tạo tài khoản tài xế      |
| POST        | /api/auth/login    | Xác thực tài xế           |
| GET         | /api/auth/me       | Lấy hồ sơ tài xế hiện tại |

## Proxy AI Agent

| Phương thức | Endpoint            | Mô tả                                   |
|-------------|---------------------|-----------------------------------------|
| POST        | /api/ai/investigate | Chuyển tiếp đến dịch vụ AI Agent Python |

## 8. Giao Thức WebSocket

Tiếp Nhận Tài Xế: `/ws/driver`

Hướng: Trình mô phỏng → Backend

Loại tin nhắn: TextMessage (JSON) hoặc BinaryMessage (Protobuf)

Payload: `GPSTBatch` — mảng các điểm GPS

```
{
  "points": [
    {
      "driver_id": "d-001",
      "trip_id": "trip-abc-123",
      "latitude": 41.1496,
      "longitude": -8.6109,
      "timestamp": 1724389200000,
      "heading": 45.0,
      "speed": 32.5
    }
  ]
}
```

Tần suất: Mỗi 3 giây từ trình mô phỏng

Ràng buộc kích thước: < 1 KB mỗi lô

Bản Đồ Nhiệt Quản Trị: `/ws/admin`

Hướng: Backend → Bảng điều khiển quản trị

Loại tin nhắn: TextMessage (JSON)

Payload: `HeatmapUpdate` — các ô đã thay đổi kể từ lần cập nhật trước (delta)

```
{
  "cells": [
    { "h3_index": "H8:2602:25773", "intensity": 3, "last_updated": 1724389200000 }
  ],
  "server_timestamp": 1724389200000,
  "total_drivers": 12,
  "total_deviations": 47
}
```

Tần suất: Mỗi 1 giây (khớp với chu kỳ flush Redis)

Ràng buộc kích thước: < 10 KB mỗi lần cập nhật

Ngoài ra, sự kiện `new_trip` được phát sóng khi trình mô phỏng lưu chuyến đi hoàn thành:

```
{
  "type": "new_trip",
  "trip": { "trip_id": "...", "driver_id": "...", "is_deviated": true, ... }
}
```

## 9. Lược Đồ Protobuf

File: `proto/heatmap/v1/messages.proto`

Đây là nguồn chính xác duy nhất cho mọi định dạng tin nhắn. Sau khi thay đổi:

- Go: `cd backend && make proto` → tạo code trong `backend/gen/heatmap/v1/`
- JS: Frontend dùng `protobufjs` để tải file `.proto` trực tiếp

Tin Nhắn: Trình Mô Phỏng → Backend

| Tin nhắn  | Trường                                                             | Mô tả                     |
|-----------|--------------------------------------------------------------------|---------------------------|
| GPSPoint  | driver_id, trip_id, latitude, longitude, timestamp, heading, speed | Một điểm GPS đọc được     |
| GPSTBatch | repeated GPSPoint points                                           | Lô gửi mỗi 3 giây         |
| TripRoute | trip_id, driver_id, repeated Waypoint waypoints                    | Đăng ký tuyến kế hoạch    |
| Waypoint  | latitude, longitude                                                | Một điểm trên tuyến đường |

Tin Nhắn: Backend → Quản Trị

| Tin nhắn      | Trường                                                                        | Mô tả                           |
|---------------|-------------------------------------------------------------------------------|---------------------------------|
| HeatmapCell   | h3_index, intensity, last_updated                                             | Một ô hexagon trên bản đồ nhiệt |
| HeatmapUpdate | repeated HeatmapCell cells, server_timestamp, total_drivers, total_deviations | Cập nhật delta mỗi 1 giây       |

10. Frontend — Bảng Điều Khiển Quản Trị

Thư mục: frontend/admin/ (React 18 + Vite)

Kiến Trúc Component

|                             |                                                      |
|-----------------------------|------------------------------------------------------|
| App.jsx                     |                                                      |
| ├─ MapContainer.jsx         | # Khởi tạo bản đồ MapLibre GL                        |
| ├─ HeatmapLayer.jsx         | # 🌟 Vẽ hexagon H3, đùn 3D, logic fetch               |
| ├─ FilterPanel.jsx          | # Bộ lọc khoảng ngày, tài xế, chuyến đi, điều tra AI |
| ├─ StatsOverlay.jsx         | # Hiển thị thống kê live/lịch sử                     |
| ├─ DriverList.jsx           | # Danh sách tài xế với click-to-filter               |
| ├─ HourlyAnalyticsChart.jsx | # Biểu đồ phân bố né tránh 24 giờ                    |
| └─ ToastNotification.jsx    | # Thông báo chuyến đi mới thời gian thực             |

Chiến Lược Tải Dữ Liệu

Tải Dữ Liệu Tiến Triển ( useProgressiveData.js ):

| Pha             | Endpoint           | Thời gian | Dữ liệu                                    |
|-----------------|--------------------|-----------|--------------------------------------------|
| Pha 1 (tức thì) | /api/stats-summary | ~300ms    | Tổng số đếm, khoảng ngày thực từ CSDL      |
| Pha 2 (nền)     | /api/trips-summary | Streaming | Danh sách chuyến đi phân trang (200/trang) |
| Pha 3 (nền)     | /api/points        | Streaming | Điểm GPS chia khúc (50k/khúc)              |

Hiển Thị Hexagon H3 ( HeatmapLayer.jsx ):

- Resolution thích ứng: Mức zoom → H3 resolution (8–14)
- Fetch theo viewport: Khi moveend, debounce (400ms) rồi fetch /api/h3-aggregate với bbox viewport hiện tại
- Mở rộng bbox: Đệm 150% mỗi hướng để tải trước hexagon gần mép viewport
- Bbox tối thiểu theo resolution: Ngăn viewport quá nhỏ trả về 0 ô ở resolution cao
- Vẽ hexagon phía client: Nhận center + cell\_size từ backend, vẽ đa giác 6 đỉnh dùng bảng tra cos/sin (~10ms cho 22k ô)
- Cache LRU: 20 mục, khóa bằng resolution:bboxHash
- Đùn 3D: Lớp fill-extrusion của MapLibre với chiều cao tỷ lệ thuận ratio

Các Chế Độ

| Chế độ    | Nguồn dữ liệu         | Mô tả                                                            |
|-----------|-----------------------|------------------------------------------------------------------|
| Lịch sử   | REST API + PostgreSQL | Duyệt hơn 386k sự kiện lịch tuyến với bộ lọc thời gian/tài xế    |
| Trực tiếp | WebSocket + Redis     | Luồng lịch tuyến thời gian thực từ trình mô phỏng đang hoạt động |

Tích Hợp OSRM (Frontend)

File: `utils/osrmRouting.js`

- `matchTripToRoads()` — OSRM `/match` API cho GPS trace → polyline bắt dính đường
- `getPlannedRoute()` — OSRM `/route` API cho polyline tuyến kế hoạch
- `computeH3Overlap()` — Tính giao nhau ô H3 giữa tuyến thực tế và kế hoạch

11. Frontend — Trình Mô Phỏng Tài Xế

Thư mục: `frontend/simulator/` (React 18 + Vite)

Tính Năng

- **Xác thực tài xế:** Modal đăng ký/đăng nhập với phiên lưu trữ bền (localStorage)
- **Lên kế hoạch tuyến:** Click điểm đi + điểm đến trên bản đồ → OSRM `/route` cho đường kế hoạch
- **Vẽ tương tác:** Vẽ tuyến tùy chỉnh trên bản đồ → OSRM `/match` để bắt dính đường
- **Mô phỏng GPS:** Web Worker tạo điểm GPS dọc tuyến kế hoạch với cấu hình:
  - Xác suất đi lệch (tần suất tài xế ra khỏi tuyến)
  - Biến thiên tốc độ
  - Số tài xế mô phỏng
- **Streaming WebSocket:** Gửi `GPSTracks` đến backend mỗi 3 giây qua `/ws/driver`
- **Lưu chuyến đi:** Lưu chuyến đi hoàn thành qua `POST /api/trips`
- **Lịch sử chuyến đi:** Hiển thị chuyến đi cũ với so sánh tuyến kế hoạch vs thực tế

Tuyến Đường Mặc Định

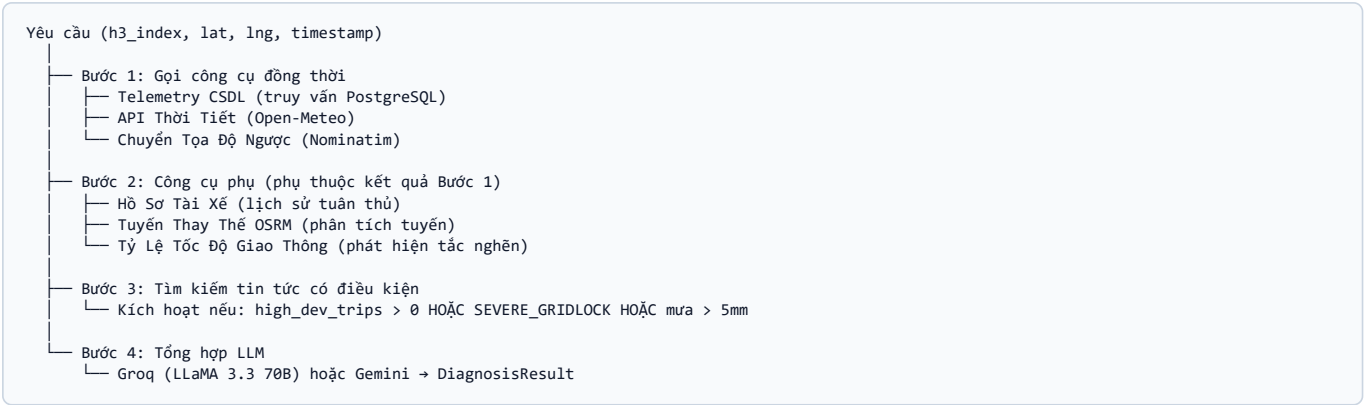
| Điểm     | Tọa độ            | Mô tả                      |
|----------|-------------------|----------------------------|
| Điểm đi  | 10.8184, 106.6588 | Sân bay Tân Sơn Nhất (SGN) |
| Điểm đến | 10.7725, 106.6980 | Chợ Bến Thành, Quận 1      |

12. Dịch Vụ AI Agent

Thư mục: `ai/agent/` (Python + FastAPI)

Kiến Trúc: Mẫu ReAct

AI agent tuân theo vòng lặp Suy Luận - Hành Động - Quan Sát:



Các Hàm Công Cụ (7 tổng cộng)

| Công cụ                          | Nguồn                    | Mô tả                                                                      |
|----------------------------------|--------------------------|----------------------------------------------------------------------------|
| <code>db_telemetry</code>        | PostgreSQL               | Truy vấn sự kiện lịch tuyến, tính thống kê, làm mượt Bayesian, ngưỡng động |
| <code>weather</code>             | Open-Meteo API           | Nhiệt độ, lượng mưa, tốc độ gió (lịch sử hoặc hiện tại)                    |
| <code>reverse_geocode</code>     | Nominatim                | Tọa độ → địa chỉ dễ đọc                                                    |
| <code>news_search</code>         | Tìm kiếm web             | Tìm sự cố giao thông gần vị trí                                            |
| <code>osrm_alternatives</code>   | OSRM <code>/route</code> | Phân tích tuyến thay thế (đi tắt vs đi vòng)                               |
| <code>driver_profile</code>      | PostgreSQL               | Tỷ lệ tuần thủ 30 ngày, mức uy tín                                         |
| <code>traffic_speed_ratio</code> | PostgreSQL + OSRM        | Tốc độ cơ sở vs hiện tại, phân loại tắc nghẽn                              |

Phân Loại Rủi Ro

| Mức                               | Ý nghĩa                                                                    |
|-----------------------------------|----------------------------------------------------------------------------|
| <code>SAFE_FORCE_MAJEURE</code>   | Lịch tuyến được giải thích bởi thời tiết, giao thông, hoặc điều kiện đường |
| <code>SUSPICIOUS</code>           | Mẫu bất thường, cần theo dõi                                               |
| <code>FRAUD_ALERT</code>          | Rủi ro cao: vi phạm lặp lại, không có giải thích từ môi trường             |
| <code>ANALYSIS_UNAVAILABLE</code> | Dữ liệu không đủ để chẩn đoán                                              |

Mô Hình Dữ Liệu

Yêu cầu: `InvestigateRequest`

```
h3_index: str          # Ô H3 cần điều tra
lat, lng: float        # Tọa độ tâm
time_window_minutes: int = 60
timestamp_ms: int | None
driver_id: str | None
end_lat, end_lng: float | None # Tọa độ điểm đến cho phân tích OSRM
```

Phản hồi: `DiagnosisResult`

```
h3_index: str
risk_level: str        # "SAFE_FORCE_MAJEURE" | "SUSPICIOUS" | "FRAUD_ALERT"
confidence: float      # 0.0 đến 1.0
summary: str           # Chẩn đoán dễ đọc (tiếng Việt)
evidence: Evidence     # Tất cả bằng chứng thu thập được
recommendation: str    # Hành động đề xuất
```

### 13. Hạ Tầng & Triển Khai

#### Các Dịch Vụ Docker Compose

File: `infra/docker-compose.yml`

| Dịch vụ               | Image                                       | Cổng          | Giới hạn RAM | Giới hạn CPU | Phụ thuộc                                                      |
|-----------------------|---------------------------------------------|---------------|--------------|--------------|----------------------------------------------------------------|
| <code>osrm</code>     | <code>osrm/osrm-backend:latest</code>       | 5000 (nội bộ) | 400 MB       | 1.0          | —                                                              |
| <code>postgres</code> | <code>postgres/postgis:16-3.4-alpine</code> | 5432          | 800 MB       | 0.5          | —                                                              |
| <code>redis</code>    | <code>redis:7-alpine</code>                 | 6379          | 200 MB       | 0.25         | —                                                              |
| <code>backend</code>  | Tùy chỉnh (Go đa giai đoạn)                 | 8080 (nội bộ) | 100 MB       | 0.5          | <code>osrm</code> , <code>postgres</code> , <code>redis</code> |
| <code>ai-agent</code> | Tùy chỉnh (Python)                          | 8090          | 150 MB       | 0.5          | <code>postgres</code>                                          |
| <code>nginx</code>    | <code>nginx:alpine</code>                   | 80            | 50 MB        | 0.25         | <code>backend</code>                                           |

Tổng ngân sách tài nguyên: 1.700 MB RAM, 3.0 lõi CPU — thiết kế cho **VPS 4 GB**.

#### Định Tuyến Nginx

| Đường dẫn                 | Backend          | Giao thức                         |
|---------------------------|------------------|-----------------------------------|
| <code>/api/*</code>       | Go backend :8080 | HTTP (timeout 30 giây)            |
| <code>/ws/*</code>        | Go backend :8080 | WebSocket upgrade (timeout 1 giờ) |
| <code>/admin/*</code>     | File tĩnh        | SPA (fallback index.html)         |
| <code>/simulator/*</code> | File tĩnh        | SPA (fallback index.html)         |
| <code>/</code>            | Chuyển hướng     | → <code>/admin</code>             |

#### Dockerfile Backend

Build đa giai đoạn:

- 1. Giai đoạn build: `golang:1.22-alpine` + `gcc musl-dev` (cho CGO)
- 2. Giai đoạn chạy: `alpine:3.19` với người dùng không phải root (uid 1001)
- 3. Bảo mật: `ca-certificates`, `libc6-compat`, chạy dưới quyền `appuser`
- 4. Health check: `wget -qO- http://localhost:8080/api/health`

#### Lệnh Khởi Động

```
# Production (toàn bộ stack Docker)
make up
# hoặc: docker compose -f infra/docker-compose.yml --env-file .env up -d --build

# Phát triển (Docker cho hạ tầng, native cho code ứng dụng)
npm run start
# Lệnh này chạy:
# 1. prestart: docker compose up postgres redis ai-agent
# 2. concurrently: go run ./cmd/server | npm run dev (admin) | npm run dev (simulator)
```

## 14. Ràng Buộc Hiệu Năng & Tối Ưu Hóa

### Ràng Buộc Backend

| Chỉ số                                     | Mục tiêu | Thực tế                                             |
|--------------------------------------------|----------|-----------------------------------------------------|
| RAM Go backend                             | < 50 MB  | Đo trong giới hạn cho 500 tài xế                    |
| Tin nhắn WebSocket (tài xế → backend)      | < 1 KB   | ~200–500 bytes mỗi lô                               |
| Cập nhật bản đồ nhiệt (backend → quản trị) | < 10 KB  | ~2–8 KB mỗi delta 1 giây                            |
| Timeout OSRM                               | 500 ms   | Cấu hình qua <code>OSRM_MATCH_TIMEOUT_MS</code>     |
| Chu kỳ flush Redis                         | 1 giây   | Cấu hình qua <code>FLUSH_INTERVAL_REDIS_MS</code>   |
| Chu kỳ flush PostgreSQL                    | 30 giây  | Cấu hình qua <code>FLUSH_INTERVAL_POSTGRES_S</code> |

### Hiệu Năng Tổng Hợp H3

| Resolution   | Số ô điển hình | Thời gian phản hồi | Yêu cầu Bbox |
|--------------|----------------|--------------------|--------------|
| 8 (mặc định) | ~1.200         | ~800ms             | Không        |
| 9            | ~9.700         | ~1,5 giây          | Không        |
| 10           | ~22.000        | ~800ms             | Có (0,20°)   |
| 11           | ~15.000        | ~600ms             | Có (0,10°)   |
| 12           | ~31.500        | ~1,2 giây          | Có (0,08°)   |
| 13           | ~4.700         | ~412ms             | Có (0,01°)   |
| 14           | ~3.600         | ~240ms             | Có (0,006°)  |

### Chiến Lược Cache (3 Tầng)

| Tầng                      | Loại cache                           | TTL     | Mục đích                                       |
|---------------------------|--------------------------------------|---------|------------------------------------------------|
| Backend h3-aggregate      | <code>sync.Map</code> (trong bộ nhớ) | 60 giây | Tránh quét lại 386k dòng khi zoom/pan nhanh    |
| Frontend h3GeoJSON        | JavaScript <code>Map</code> (LRU-20) | Phiên   | Vẽ lại tức thì khi zoom ngược lại              |
| Frontend progressive data | <code>sessionStorage</code>          | 5 phút  | Tránh fetch lại trips/points khi tải lại trang |

### Kỹ Thuật Tối Ưu Hóa

- Lọc Trước Hộp Bao:** ~80% điểm GPS không bao giờ đến OSRM (kiểm tra chứa  $O(1)$ )
- Bộ Tổng Hợp Không Khóa:** `sync.Map` + `atomic.AddUint64` — không tranh chấp mutex trên đường nóng
- H3 Phía Server:** Loại bỏ việc gửi 386k điểm thô đến trình duyệt; nhanh hơn 50–300 lần
- Nén Gzip:** Giảm ~80% dung lượng phản hồi JSON (quan trọng cho payload 30k ô)
- Fetch Theo Viewport:** Frontend chỉ fetch vùng nhìn thấy + đệm 150%
- Resolution Thích Ứng:** Zoom cao hơn → ô nhỏ hơn → bbox nhỏ hơn → ít dòng quét hơn
- Chỉ Mục PostgreSQL:** 7 chỉ mục bao phủ mọi mẫu truy vấn (BTree, BRIN, composite)
- Hexagon Tính Trước:** Backend gửi `center_lat + cell_size`; frontend vẽ bằng bảng tra sin/cos

## 15. Biến Môi Trường

Xem `.env.example` để có mẫu đầy đủ với chú thích.

Bắt Buộc Cho Production

| Biến              | Ví dụ                  | Mô tả                      |
|-------------------|------------------------|----------------------------|
| POSTGRES_PASSWORD | (thay đổi từ mặc định) | Mật khẩu cơ sở dữ liệu     |
| GROQ_API_KEY      | gsk_...                | Khóa API Groq cho AI agent |

Biến Frontend Lúc Build

| Biến           | Mặc định                  | Mô tả              |
|----------------|---------------------------|--------------------|
| VITE_WS_URL    | ws://localhost:8080/ws    | URL gốc WebSocket  |
| VITE_API_URL   | http://localhost:8080/api | URL gốc REST API   |
| VITE_MAP_STYLE | URL CARTO dark-matter     | Kiểu tile MapLibre |

16. Quy Trình Phát Triển

Bắt Đầu Nhanh

```
# 1. Clone và cài đặt
git clone <repo_url>
cd heat_map_pro
npm run install:all

# 2. Sao chép cấu hình môi trường
cp .env.example .env

# 3. Khởi động mọi thứ (Docker hạ tầng + ứng dụng native)
npm run start
```

Lệnh này sẽ:

- Khởi động PostgreSQL, Redis, AI Agent qua Docker Compose
- Khởi động Go backend ( `go run ./cmd/server` )
- Khởi động Bảng điều khiển quản trị (Vite dev server, cổng 3002)
- Khởi động Trình mô phỏng (Vite dev server, cổng 3001)

Các Lệnh Make Có Sẵn

| Lệnh                           | Mô tả                                                    |
|--------------------------------|----------------------------------------------------------|
| <code>make up</code>           | Khởi động toàn bộ stack Docker production                |
| <code>make down</code>         | Dừng tất cả dịch vụ Docker                               |
| <code>make dev</code>          | Khởi động môi trường dev với <code>air</code> hot-reload |
| <code>make proto</code>        | Tạo lại code Go từ Protobuf                              |
| <code>make test-all</code>     | Chạy tất cả test Go với cờ <code>-race</code>            |
| <code>make lint</code>         | Chạy <code>golangci-lint</code> trên backend             |
| <code>make osrm-prepare</code> | Tải và chuẩn bị dữ liệu bản đồ OSRM                      |
| <code>make clean</code>        | Xóa tất cả artifact build                                |

Quy Ước Git

- Đặt tên nhánh: `feature/<module>-<mô_tả>` , `fix/<module>-<mô_tả>`
- Thông điệp commit: `feat(module): mô tả` , `fix(module): mô tả`

## 17. Chiến Lược Kiểm Thử

### Backend Go

- **Lệnh:** `go test ./... -v -race -count=1`
- **Mẫu:** Test theo bảng (table-driven) trong file `_test.go` cạnh source code
- **Phạm vi bao phủ:**
  - `aggregator/lockfree_test.go` — Tăng đồng thời + tính đúng đắn snapshot
  - `aggregator/aggregator_test.go` — Logic tổng hợp chung
  - `filter/bounding_box_test.go` — Độ chính xác kiểm tra chứa
  - `filter/osrm_client_test.go` — Mock HTTP server cho phản hồi OSRM
  - `filter/filter_test.go` — Tích hợp: bbox → OSRM → ngưỡng pipeline
  - `spatial/h3_indexer_test.go` — Tính toán lưới, khứ hồi ô ↔ tọa độ
  - `persistence/postgres_test.go` — Thao tác cơ sở dữ liệu

### AI Agent

- **File:** `ai/agent/test_suite.py`
- **Test:** Unit test hàm công cụ, kiểm tra định dạng phản hồi LLM, tích hợp ReAct engine

### Frontend

- React Testing Library cho test hiển thị component (dự kiến)
- Kiểm tra thủ công: quan sát trực quan hiển thị H3 ở nhiều mức zoom

### Kiểm Thử Tích Hợp

- Client OSRM được test với mock HTTP server (không phải OSRM thật)
- Test pipeline đầy đủ: mô phỏng điểm GPS → xác nhận sự kiện lệch tuyến trong PostgreSQL

Tài liệu được tạo từ phân tích mã nguồn kho `heat_map_pro`.  
Tất cả tham chiếu code đều liên kết đến file nguồn thực tế để truy vết.